06 堆積排序 解答

06-E01 max-heapify 追蹤

推導

根 4 與子 9、8 比較，最大為 9，交換得 [9,4,8,3,7,6,5]。位置 2 的 4 與子 3、7 比較，7 較大，交換得 [9,7,8,3,4,6,5]。位置 5 已無子節點，停止。

結論與檢核

最終各父子關係都滿足 max-heap。執行路徑只能沿被交換子樹下降，不會修復另一棵未受影響子樹。

06-E02 線性建堆的節點高度和

推導

高度 0 的葉有 8 個，成本 0；高度 1 有 4 個，成本 4；高度 2 有 2 個，成本 4；高度 3 有 1 個，成本 3。總和為 11，是小於 c*15 的常數倍。

結論與檢核

對一般n，至少h高的節點數恰為floor(n/2^h)，所以高度總和sum_i height(i)=sum_(h>=1)floor(n/2^h)<n。再加每次呼叫常數成本，總O(n)。這也直接處理不滿的最後一層，不忽略ceil造成的項。

06-E03 優先權更新

推導

位置 8 的父位置是 4，值 7；16>7，交換後位置 4 為 16。新父位置 2 為 15，16>15，再交換。位置 2 的父位置是根 20，16<20，停止。

結論與檢核

最終陣列為 [20,16,18,15,12,9,17,7]。舊值 2 已被 16 取代，不能仍保留在陣列末尾。每次上移只把違規移向父邊，其他子樹不變；若鍵下降則需 downward repair。

06-E04 Heap驗證器的奇數Child反例

推導

A的兩個child都接在根10下，10>=1且10>=9，所以本來是合法heap。錯誤規則在c3用ceil(3/2)=2，要求A2=1>=A3=9，錯拒合法輸入。

06 堆積排序 解答 （續）

結論與檢核

1-index須用floor(c/2)，對所有c2..n檢查父鍵。0-based child j1..n−1的父是floor((j−1)/2)，檢查a[(j−1)//2]>=a[j]。不能把兩套索引混在同一式子；只通過某個大根範例不足以驗證檢查器正確。

06-E05 相等鍵的穩定性追蹤

推導

建堆已合法。先根與末端交換，得(1,c),(2,b),(2,a)。heap-size2，heapify把前兩項交換成(2,b),(1,c),(2,a)。再交換前兩項並縮成size1，得(1,c),(2,b),(2,a)。

結論與檢核

鍵已排序但同鍵原順序a,b變b,a，故不穩定。只檢查鍵序與multiset無法驗證穩定性，還須追蹤identity；若另把原序號加入比較鍵，那是額外規格與儲存成本，不是原heapsort自動穩定。

06-E06 指定位置刪除的兩種修復方向

推導

刪位置4的10，末43補到4，父位置2為20，43>20，上浮交換4、2，到根50停止，得[50,43,45,20,15,40,44,5,9,12,14,35,39,42]。獨立刪根50，末43補根，先與較大child45交換，再在位置3與child44交換，得[45,20,44,10,15,40,43,5,9,12,14,35,39,42]。

結論與檢核

若替補鍵大於父，因原父>=原被刪鍵>=其children，替補鍵也大於原children，只可能向上違規。否則父邊已合法，只需向下heapify。每步沿單一路徑，最壞O(log n)；找出位置本身不包含在此界內，若外界以handle指定，swap時必更新handle-to-index對照。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

06 堆積排序 解答 （續）

原創驗算程式附錄: heap_build_trace.py

```
"""Reproduce the ten-element linear-time heap construction trace."""


def heapify(a, i, size=None):
    size=len(a) if size is None else size
    while True:
        left = 2 * i + 1
        right = left + 1
        largest = i
        if left < size and a[left] > a[largest]:
            largest = left
        if right < size and a[right] > a[largest]:
            largest = right
        if largest == i:
            return
        a[i], a[largest] = a[largest], a[i]
        i = largest


def build_heap(a):
    a = list(a)
    stages = []
    for one_based in range(len(a) // 2, 0, -1):
        heapify(a, one_based - 1)
        stages.append((one_based, tuple(a)))
    return a, stages


def is_max_heap(a):
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

06 堆積排序 解答 （續）

```
    return all(
        a[(child - 1) // 2] >= a[child]
        for child in range(1, len(a))
    )


def heap_sort_trace(values):
    a,_=build_heap(values)
    rows=[]
    for end in range(len(a)-1,0,-1):
        a[0],a[end]=a[end],a[0]
        heapify(a,0,end)
        assert is_max_heap(a[:end])
        assert a[end:]==sorted(a[end:])
        assert max(a[:end])<=a[end]
        rows.append((a[:end],a[end:]))
    assert a==sorted(values)
    return rows


def increase_key(a,position,key):
    i=position-1
    assert 0<=i<len(a) and key>=a[i]
    a[i]=key
    while i>0:
        parent=(i-1)//2
        if a[parent]>=a[i]:
            break
        a[parent],a[i]=a[i],a[parent]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

06 堆積排序 解答 （續）

```
        i=parent
    return a


def delete_at(a,position):
    i=position-1
    assert 0<=i<len(a)
    removed=a[i]
    last=a.pop()
    if i<len(a):
        a[i]=last
        if i>0 and a[i]>a[(i-1)//2]:
            increase_key(a,i+1,a[i])
        else:
            heapify(a,i)
    return removed


def run():
    source = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]
    expected = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]
    result, stages = build_heap(source)
    assert result == expected
    assert [i for i, _ in stages] == [5, 4, 3, 2, 1]
    assert stages[3][1] == (4, 16, 10, 14, 7, 9, 3, 2, 8, 1)
    assert is_max_heap(result)
    assert is_max_heap([10,1,9])
    assert not is_max_heap([10,11,9])
    sort_rows=heap_sort_trace(source)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

06 堆積排序 解答 （續）

```
    assert sort_rows[1]==([10,8,9,4,7,1,3,2],[14,16])
    updated=increase_key([20,15,18,7,12,9,17,2],8,16)
    assert updated==[20,16,18,15,12,9,17,7] and is_max_heap(updated)
    before=[50,20,45,10,15,40,44,5,9,12,14,35,39,42,43]
    up=before[:]
    assert delete_at(up,4)==10
    assert up==[50,43,45,20,15,40,44,5,9,12,14,35,39,42] and is_max_heap(up)
    down=before[:]
    assert delete_at(down,1)==50
    assert down==[45,20,44,10,15,40,43,5,9,12,14,35,39,42] and is_max_heap(down)
    from itertools import product
    cases=0
    for length in range(7):
        for values in product(range(3),repeat=length):
            built,_=build_heap(values)
            assert is_max_heap(built)
            assert sorted(built)==sorted(values)
            heap_sort_trace(values)
            cases+=1
    print(f"PASS heap_valid_odd_child_counterexample=True exhaustive_small_arrays={cases}
increase_key={updated} heapsort_rows={sort_rows}")
    print("PASS heap-build stages=5 arrays=5 final=" + str(result))
    return result


if __name__ == "__main__":
    run()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

06 堆積排序 解答 （續）

原創驗算程式附錄: heap_operations.py

```
"""Build a max-heap and verify all parent-child relations."""

def max_heapify(a, i, size):
    while True:
        left, right = 2 * i + 1, 2 * i + 2
        largest = i
        if left < size and a[left] > a[largest]:
            largest = left
        if right < size and a[right] > a[largest]:
            largest = right
        if largest == i:
            return
        a[i], a[largest] = a[largest], a[i]
        i = largest


def build_heap(a):
    for i in range(len(a) // 2 - 1, -1, -1):
        max_heapify(a, i, len(a))


sample = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]
build_heap(sample)
assert all(sample[i] >= sample[2 * i + 1] for i in range(len(sample) // 2))
assert all(2 * i + 2 >= len(sample) or sample[i] >= sample[2 * i + 2] for i in
range(len(sample) // 2))
print(sample)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study